Process exercises

Identify state, assumptions, and intermediate steps before reading the separate solutions.

OS03-E01 State transitions

On one CPU, P1 starts a disk read while P2 is ready. A completion interrupt occurs later.

1. Trace P1/P2 states.
2. Where can a context switch occur?
3. Must P1 run immediately after the interrupt?

OS03-E02 Zombie and orphan

A server forks a child that calls exec. Separately, a parent exits while its child still runs.

1. Which condition produces a zombie?
2. Which produces an orphan?
3. How can the server avoid indefinite zombie accumulation?

OS03-E03 Bounded message channel and termination

A custom FIFO has two whole-message slots and atomic send/receive of one message. There is one producer/consumer; positive jobs and STOP=0. This is not a claim about native pipe/queue APIs.

1. What happens when J3 follows queued J1/J2?
2. Can this channel short-write half a message?
3. How can it drain and stop? What if two consumers receive only one STOP?

OS03-E04 Context and object identity

P100 has PC40/r1=7/SP9000; P101 has saved PC120/r1=3/SPA000, addresses in hex. They share a code image but fd3 references different descriptions.

1. What is saved/restored during a switch?
2. Do equal image/VA/fd3 imply equal data?
3. Does a syscall necessarily switch process/AS?

OS03-E05 Lost wakeup

Initially done=false. An incorrect waiter checks false before enqueueing/sleeping. The handler wakes only registered waiters.

1. Give an indefinitely blocked interleaving.
2. Repair it with one lock/state protocol and explain completion timing cases.
3. Is a woken task immediately running?

OS03-E06 Fork tree and lifecycle

Two unconditional forks succeed with no early exit. P0 initially has fd3→O1.offset 0. Separately, a child successfully execs worker and exits 7.

1. Name the four processes and premises for 2^k.
2. The file is long enough. The parent successfully reads 100 bytes, then the child successfully reads 50 bytes, with no intervening operations; both share O1. Give the offsets before and after each read.
3. Explain exec, zombie, orphan, and wait status.

OS03-E07 SPSC ring and publication

Three slots, head/tail 0, one reserved slot, one producer/consumer. Put A/B, try C, get A, put C, get B/C.

1. Trace head, tail, and FIFO.
2. What do the two release/acquire relationships protect?
3. Can plain integers or two producers use the same proof?

OS03-E08 Framing and shared objects

Stream 00 03 41 42 43 00 02 44 45 uses 2 B big-endian lengths, max 8 B payload. A shared object maps at hex 100000 and 700000.

1. Reads are[00],[03 41 42],[43 00 02 44 45]. When are records emitted?
2. Handle EOF after the fourth byte and a length 9.
3. Give each VA for offset 16 and explain unlink/close.
4. Are POSIX mq_unlink and System V IPC_RMID equivalent to draining?

OS03-E09 RPC retry and encoding

Balance 100, client A request 77 debits 25, and reply may be lost. Wire id 32/op 16/amount 32 is big-endian; debit opcode 1.

1. Encode the request and give length.
2. Give two timeout histories and the blind-retry problem.
3. How can duplicate and concurrent same-ID requests avoid repeated effects?
4. What fails with RAM records, separate writes, or early record expiry?

OS03-E10 Seven-state event order

A runs in memory, starts I/O, and is swapped out. I/O completes while A remains on backing storage; A is then swapped in and selected by the dispatcher. List the states. What changes if A is swapped in before I/O completes?

OS03-E11 A race after the pre-sleep check

The waiter has registered and, after releasing the lock, observes WAITING and prepares to sleep. Completion sends exactly one wakeup. Give an interleaving that still loses the wakeup and state the necessary wait primitive guarantee.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 3. read(2); pipe(7); msgctl(2); mq_unlink(3).

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study